

Vivado Simulation Tutorial

1 Purpose

The purpose of this laboratory work is to get familiar with the Vivado Design Suite and with the functional (behavioral) simulation of digital circuits described in VHDL. A 1-bit full adder is used as a running example: a Vivado project targeting the Basys 3 development board is created, the full adder is described in VHDL, a testbench is written and the design is simulated using the Vivado integrated simulator.

2 Introduction

In order to design digital systems using programmable logic devices such as FPGAs, computer aided design (CAD) software products are used. These software solutions offer the whole package, from the early designing stage to the final FPGA implementation stage. Thus, 5 steps can be distinguished:

1. **Description:** VHDL source code
2. **Synthesis:** compile VHDL code, transform description into a netlist (basic logic gates and interconnections)
3. **Functional simulation:** simulation based on test benches and waveforms (usually from the IDE, such as Vivado Simulator, Xilinx ISE Simulator, ModelSim etc.)
4. **Implementation:** adapting the netlist to the available resources of the device (technology mapping, place and route)
5. **Configuration:** programming the device

This laboratory focuses on the first and the third steps: **description** and **functional simulation**. Simulation allows the designer to verify that a circuit behaves as expected *before* it is synthesized and uploaded on a real device. Finding and fixing errors in the simulator is much faster and easier than debugging the design directly on the board, so every module should be simulated before it is integrated in a larger design.

The examples in this laboratory work are customized for the Basys 3 FPGA board, which is built around a Xilinx Artix-7 FPGA (**xc7a35tcpg236-1**). The board will be used in the following laboratories; for now, it is only necessary to select the correct device when creating the project.

3 Full Adder

This circuit is the basic addition block, which adds three 1-bit inputs: 2 bits to be added (x_i and y_i) and a 1 bit carry-in (C_i). It generates 2 outputs: the sum bit (S_i) and the carry-out (C_{i+1}) bit. The figure below illustrates a full adder symbol.

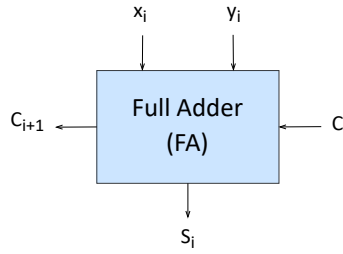


Figure 1: Full adder I/O

x_i	y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Full adder truth table

Given the truth table above, the Boolean expressions of the outputs (after reduction) are:

$$S_i = x_i \oplus y_i \oplus C_i$$

$$C_{i+1} = x_i \cdot y_i + (x_i + y_i) \cdot C_i$$

A **half adder** is basically a full adder without the carry input and it generates a sum bit and a carry bit.

In the VHDL code of this laboratory, the full adder signals are named as follows: $x_i \rightarrow \mathbf{x}$, $y_i \rightarrow \mathbf{y}$, $C_i \rightarrow \mathbf{cin}$, $S_i \rightarrow \mathbf{s}$ and $C_{i+1} \rightarrow \mathbf{cout}$.

4 Vivado Simulation Guide

This section provides a step-by-step guide for creating a Vivado project targeting the Basys 3 board, describing the full adder in VHDL, writing a testbench for it and running a behavioral simulation.

Step 1. Create a new RTL project using Vivado IDE targeting the Basys 3 device

Open Vivado, click on *File* $\rightarrow$ *Project* $\rightarrow$ *New...* and in the opened window click *Next*. In the *Project Name* window, type a name for the project (e.g. **full_adder**), provide the path or browse to the directory where you want to create the project, make sure that *Create project subdirectory* is checked and click *Next*.

Note. Avoid spaces and special characters in both the project name and the project path, as they can cause errors in the Vivado tools.

In the *Project Type* window select *RTL Project* and tick the *Do not specify sources at this time* option, then click *Next*. In the *Default Part* window, the device properties that you need to choose are:

- **Product category:** All
- **Family:** Artix-7
- **Package:** cpg236

- **Speed grade:** -1

Then select from the table the **xc7a35tcbg236-1** device and click *Next*. Check the summary in the *New Project Summary* window and click *Finish*.

After the project is created, the main Vivado window is displayed. Take a moment to identify its main areas:

- **Flow Navigator** (left side) – gives access to all design steps: *Project Manager*, *Simulation*, *RTL Analysis*, *Synthesis*, *Implementation* and *Program and Debug*;
- **Sources** tab – shows the hierarchy of the project files, grouped in *Design Sources*, *Constraints* and *Simulation Sources*;
- **Editor** area – used to view and edit the source files;
- **Tcl Console**, **Messages**, **Log** and **Reports** tabs (bottom side) – display the messages, warnings and errors generated by the tools.

Step 2. Describe the 1-bit full adder in VHDL

In the *Sources* tab, right-click on *Design Sources* and then click on *Add Sources....* In the newly opened window select *Add or create design sources* and click on *Next* button. Then select *Create File*, type `full_adder` in the *File name* text field, make sure that the selected *File type* is VHDL and press *OK*. Then, click on *Finish* button.

In the *Define Module* window specify the input and output ports of the full adder, according to Section 3:

- `x`, `y`, `cin` – direction *in*, no *Bus*;
- `s`, `cout` – direction *out*, no *Bus*.

Press *OK*. Alternatively, you can leave the ports undefined for the moment and manually define them in the created `.vhd` file.

Open the `full_adder.vhd` file by double-clicking on it in the *Sources* tab. Vivado has already generated the library declarations, the entity with the specified ports and an empty architecture. Rename the generated architecture from **Behavioral** to **Dataflow** (the architecture name is chosen by the designer and usually indicates the description style) and complete it using the logic equations of the full adder. The complete content of the file is:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port ( x      : in  STD_LOGIC;
          y      : in  STD_LOGIC;
          cin    : in  STD_LOGIC;
          s      : out STD_LOGIC;
          cout   : out STD_LOGIC);
end full_adder;

architecture Dataflow of full_adder is
begin
    s <= x xor y xor cin; -- sum bit
    cout <= (x and y) or ((x or y) and cin); -- carry-out bit
end Dataflow;
```

The architecture above uses **concurrent signal assignments** (the dataflow description style): both statements are evaluated in parallel, every time one of the signals on the right-hand side changes its value. This reflects the behavior of the hardware, where all the logic gates of the full adder operate at the same time.

Save the file (*Ctrl+S*). Vivado automatically checks the syntax of the saved file; if there are any errors, they are underlined in the editor and listed in the *Syntax Error Files* folder of the *Sources* tab and in the *Messages* tab.


```

        cout : out STD_LOGIC);
end component;

-- signals connected to the inputs of the UUT (initialized)
signal x    : STD_LOGIC := '0';
signal y    : STD_LOGIC := '0';
signal cin  : STD_LOGIC := '0';
-- signals connected to the outputs of the UUT
signal s    : STD_LOGIC;
signal cout : STD_LOGIC;

-- duration of each input combination
constant T : time := 10 ns;

begin

    -- instantiate the UUT and map its ports to the internal signals
    uut: full_adder port map (
        x    => x,
        y    => y,
        cin  => cin,
        s    => s,
        cout => cout
    );

    -- stimuli: concurrent assignments that apply all 8 input combinations
    x <= '0', '1' after 4 * T;
    y <= '0', '1' after 2 * T, '0' after 4 * T, '1' after 6 * T;
    cin <= '0', '1' after T, '0' after 2 * T, '1' after 3 * T,
           '0' after 4 * T, '1' after 5 * T, '0' after 6 * T,
           '1' after 7 * T;

end Tb;

```

Notes.

- Each stimulus is a single concurrent signal assignment with a list of **waveform elements** separated by commas. Every element has the form **value after delay**, where the delay is measured from the beginning of the simulation (time 0), so the delays must be written in increasing order. The first element has no delay and gives the value at time 0.
- The assignments are executed only once, at the beginning of the simulation, because their right-hand sides contain no signals whose changes could re-trigger them. After the last element, each signal keeps its last value until the end of the simulation.
- The inputs follow the order of the rows in the truth table: **cin** changes every T, **y** every 2 * T and **x** every 4 * T. Declaring the time step as the constant T makes it easy to change the duration of all the combinations at once.
- **after** clauses are ignored by synthesis; they are meaningful only in simulation files.
- The names of the internal signals do not have to be the same as the names of the UUT ports, but using the same names makes the testbench easier to read.

Save the file. In the *Sources* tab, the **full_adder** entity should now appear under the **Tb_full_adder** entity in the *Simulation Sources* → *sim_1* hierarchy, with the instance name **uut**. If it is displayed with a question mark, the component declaration does not match the entity (e.g. a misspelled name).

Step 4. Configure and run the behavioral simulation

Right-click on the **Tb_full_adder.vhd** file in *Simulation Sources* → *sim_1* within the *Sources* tab and select **Set as Top** (it is displayed in bold if it is already the top module).

By default, Vivado automatically runs the simulation for 1000 ns as soon as the simulator starts. In this laboratory, the simulation will be advanced manually, step by step, in order to observe how the outputs change for each input combination. Therefore, configure the simulator so that it does not advance the simulation time when it starts: go to *Flow Navigator* → *Project Manager* → *Settings*,

select *Simulation* and, in the *Simulation* tab, set `xsim.simulate.runtime*` to `0ns`. Click on *Apply* and then on *OK*.

In the *Simulation* section of the *Flow Navigator* click on *Run Simulation* → *Run Behavioral Simulation*. Vivado will compile the VHDL files, elaborate the design and start the integrated simulator (XSim). If there are compilation errors, they are listed in the *Messages* tab and the simulator does not start; fix them and run the simulation again.

Step 5. Analyze the simulation waveform

When the simulator starts, the *Simulation* perspective is opened. It contains the following windows:

- **Scope** – the hierarchy of the simulated design (`Tb_full_adder` and its instance `uut`);
- **Objects** – the signals of the scope selected in the *Scope* window;
- **Waveform window** – the evolution in time of the signals;
- **Tcl Console** – the messages of the simulator; commands can also be typed here.

By default, the waveform window contains all the signals of the top-level testbench. Because the run time was set to 0 ns, the simulation is stopped at time 0 and the waveforms are still empty. To advance the simulation gradually, use the *Run For* command from the simulation toolbar (at the top of the Vivado window):

1. In the text field next to the *Run For* button (the play button with a *T* subscript), type `10`, and in the drop-down list next to it select the time unit `ns` (the default value is `10 μs`, which would run the whole testbench at once).
2. Click the *Run For* button. The simulation advances by 10 ns and the waveform window displays the first input combination (`x = 0`, `y = 0`, `cin = 0`) together with the resulting values of `s` and `cout`. The current simulation time is displayed in the toolbar and in the *Tcl Console*, as `run 10 ns`.
3. Click *Run For* again. Each click advances the simulation by another 10 ns, which corresponds to exactly one input combination (one row of Table 2), because the testbench changes the inputs every $T = 10$ ns. Before advancing, check that the values of `s` and `cout` are the expected ones for the current combination.
4. After 8 clicks, the simulation time reaches 80 ns and all the input combinations have been applied. Click on the *Zoom Fit* button of the waveform toolbar to see the whole simulation interval.

The same can be achieved by typing `run 10 ns` in the *Tcl Console* and pressing *Enter* for each step. To start again from time 0, press the *Restart* button (or type `restart` in the *Tcl Console*) and then continue with *Run For*.

After the 80 ns simulation, the waveform should look like in Figure 3.

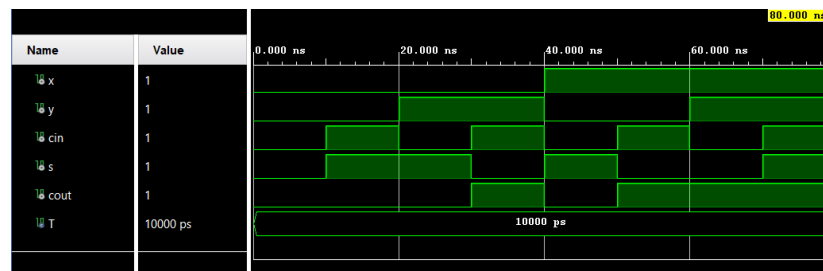


Figure 3: Waveform corresponding to the 1-bit full adder behavioral simulation

Verify that, for each 10 ns interval, the values of `s` and `cout` match the values in Table 2, which correspond to the truth table in Table 1. You can click inside the waveform to place the yellow cursor at a certain moment of time; the values of all signals at that moment are displayed in the *Value* column.

Useful operations in the simulation perspective:

Time interval	x	y	cin	s	cout
0 – 10 ns	0	0	0	0	0
10 – 20 ns	0	0	1	1	0
20 – 30 ns	0	1	0	1	0
30 – 40 ns	0	1	1	0	1
40 – 50 ns	1	0	0	1	0
50 – 60 ns	1	0	1	0	1
60 – 70 ns	1	1	0	0	1
70 – 80 ns	1	1	1	1	1

Table 2: Expected simulation results for the `Tb_full_adder` testbench

- **Adding internal signals to the waveform:** select `uut` in the *Scope* window, then, in the *Objects* window, right-click on a signal and choose *Add to Wave Window*. Signals added this way have no history, so the simulation must be restarted and run again to display their values.
- **Simulation control:** the toolbar at the top of the window (and the *Run* menu) provides the *Restart* button (resets the simulation time to 0), *Run All* (runs until there are no more events or the simulation is stopped), *Run For* (runs for the time specified in the text field next to it, e.g. 10 ns) and *Relaunch Simulation*.
- **Tcl commands:** the same actions can be performed from the *Tcl Console*, e.g. `restart` followed by `run 80 ns`.
- **Radix:** for buses (`STD_LOGIC_VECTOR` signals), right-click on the signal and choose *Radix* to display the value in *Binary*, *Hexadecimal*, *Unsigned Decimal* etc.
- **Modifying the sources:** after changing a VHDL file (design or testbench), save it and press the *Relaunch Simulation* button in the toolbar. The files are recompiled and the simulation is restarted without closing the waveform window.
- **Saving the waveform configuration:** press `Ctrl+S` in the waveform window to save the list of displayed signals in a `.wcfg` file, so that it is reloaded the next time the simulation is started.

To close the simulator, click on the *X* in the top-right corner of the blue *SIMULATION* banner, or use *Flow Navigator* → *Simulation* → *Close Simulation*.

5 Exercises

1. Follow the steps in Section 4 to create the Vivado project, describe the 1-bit full adder and simulate it using the testbench. Verify that the simulation results match Table 2.
2. Write a second architecture for the `full_adder` entity following a behavioral design (using a `process` with the sensitivity list (`x`, `y`, `cin`)) and a `case` statement, which implements directly the truth table in Table 1. Simulate it with the same testbench and compare the waveforms with the ones obtained for the first architecture.
3. Using 4 instances of the full adder, design and implement a 4-bit ripple carry adder (see Figure 4) with the following ports
 - `x`, `y` : in `STD_LOGIC_VECTOR(3 downto 0)`
 - `cin` : in `STD_LOGIC`
 - `s` : out `STD_LOGIC_VECTOR(3 downto 0)`
 - `cout` : out `STD_LOGIC`

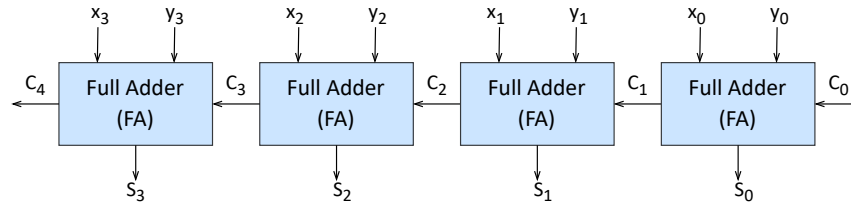


Figure 4: Design of the 4-bit Ripple-Carry Adder

Write a testbench, in the same style as `Tb_full_adder`, that applies several relevant input combinations and verify the results in the waveform window. Display `x`, `y` and `s` in the waveform window using the *Unsigned Decimal* radix.

References

- [1] Digilent Basys 3™ FPGA Board Reference Manual [accessed Sep. 2026], <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>
- [2] AMD, Vivado Design Suite User Guide: Logic Simulation (UG900) [accessed Sep. 2026], <https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation>
- [3] AMD, Vivado Design Suite Tutorial: Logic Simulation (UG937) [accessed Sep. 2026], <https://docs.amd.com/r/en-US/ug937-vivado-design-suite-simulation-tutorial>
- [4] VHDL and FPGA terminology - Testbench [accessed Sep. 2026], <https://vhdlwhiz.com/terminology/testbench/>